LARS JANSEN

Senior Software Engineer

Assen, Drenthe, Netherlands | +31 97058049397 | larsjansen0521@outlook.com | linkedin.com/in/lars-jansen-0a37b8393

PROFESSIONAL SUMMARY

Senior AI-focused Software Engineer with 9+ years delivering **production ML**, **LLM apps**, and **cloud-native** platforms across SaaS and data-driven products, turning ambiguous ideas into measurable outcomes with reliable delivery. I build end-to-end systems from **dataset curation** to **model serving** and **monitoring**, and I'm comfortable owning on-call quality, latency budgets, and cost controls. I've shipped **RAG** search experiences, tuned ranking and prompt flows, and hardened APIs with **RBAC**, **JWT**, and audit trails for enterprise clients. In US/EU distributed teams I communicate clearly, write strong docs, and unblock others through reviews and pragmatic architecture. Experienced with **Python 3.8–3.12** and modern **PyTorch 1.10–2.x** stacks in one line of work, with a bias for observability-first delivery.

SKILLS

- **GenAI / ML: Python 3.8–3.12**, **PyTorch 1.10–2.x**, **TensorFlow 2.x**, **scikit-learn 0.22–1.4**, **FastAPI 0.80–0.110**, Transformers, **RAG**, embeddings, evaluation harnesses, prompt tooling, feature engineering
- **Data / Platforms: PostgreSQL 12–16** (incl. **pgvector**), MySQL 5.7–8.0, Redis 5–7, S3 data lakes, parquet, OpenSearch/Elasticsearch 7–2.x, batch/stream processing, data quality checks
- **Cloud / DevOps:** AWS (S3, IAM, ECS/EKS, Lambda, API Gateway, SQS/SNS, CloudWatch), Docker 19–25, **Kubernetes 1.18–1.29**, Helm 3.x, Terraform 0.12–1.6, GitHub Actions, CI/CD release automation
- **MLOps / Observability / Security: MLflow 1.x–2.x**, Airflow 1.10–2.x, model registry, canary deploys, **OpenTelemetry 1.x**, Prometheus 2.x, Grafana 8–10, Datadog, structured logging, SAST/Secrets scanning, OAuth2/OIDC, **JWT**, rate limiting, audit logging

WORK HISTORY

Senior Software Engineer
RavenTrack — August 2021 to November 2025

- Led delivery of **LLM-based** support automation where a noisy retrieval layer caused hallucinations, and fixed it by adding **hybrid search**, dedupe, and eval gates that stabilized answer quality.
- Built a production **RAG** service in **FastAPI 0.9x** with **PostgreSQL 14–16 + pgvector**, implementing chunking, embeddings refresh jobs, and backpressure to protect downstream OpenSearch during spikes.
- Migrated a legacy inference worker from ad-hoc scripts to **Docker** + **Kubernetes 1.2x**, resolving "works-on-my-machine" CUDA/runtime drift by pinning images and enforcing reproducible build pipelines.
- Shipped model-serving endpoints with strict **SLOs**, using async request batching and cache keys in **Redis 6–7** to cut p95 latency and prevent thundering-herd retries on popular prompts.
- Diagnosed a nasty training regression caused by silent schema changes in parquet exports, then enforced **data contracts**, validation checks, and automated rollback when drift thresholds were exceeded.
- Implemented **MLflow 2.x** experiment tracking and a model registry workflow, so rollouts could be promoted with traceable metrics instead of "latest model wins" guesswork.
- Built an evaluation harness comparing prompts, retrievers, and rerankers, and surfaced results in dashboards so product decisions were tied to **offline + online** quality signals.
- Resolved intermittent 502s on the API edge by tracing with **OpenTelemetry 1.x**, finding a mis-sized connection pool, and tuning timeouts plus circuit breakers for downstream LLM providers.
- Introduced **canary deploys** for model versions, catching tokenization incompatibilities early and preventing wide production impact during provider SDK upgrades.
- Hardened the platform with **RBAC**, **JWT** validation, and audit logs, then added rate limiting and abuse detection to protect LLM endpoints from costly prompt floods.
- Collaborated with US teams in a follow-the-sun cadence, writing crisp RFCs and running incident reviews that converted production failures into prioritized engineering work.
- Implemented feature flags and safe rollout plans for new generative features, ensuring UX experiments didn't compromise compliance and logging requirements.
- Improved dataset refresh pipelines using partitioned loads and incremental backfills, preventing multi-hour rebuilds when only small slices of data changed.
- Built automated alerts for embedding freshness, retrieval recall, and cost-per-answer metrics, so operational health included **quality** and **spend** not just uptime.

Software Engineer
Infinite Loop — June 2018 to July 2021

- Developed scalable ML pipelines in **Python 3.7–3.10** where training jobs failed unpredictably, and fixed root causes by isolating dependency conflicts and enforcing deterministic seeds.
- Deployed real-time scoring services with **FastAPI** and background workers, optimizing serialization and payload sizes after discovering timeouts from oversized feature vectors.
- Migrated a monolith analytics service to containerized workloads using **Docker 19–20** and **Kubernetes 1.16–1.21**, improving release frequency while reducing rollout risk.

- Built NLP classification features using **PyTorch 1.6–1.10**, adding calibration and thresholding after user feedback showed high confidence on ambiguous inputs.
- Solved a production memory leak in a long-running consumer by profiling allocations, then rewriting hot paths and adding graceful restarts with idempotent message handling.
- Integrated AWS components like **SQS/SNS** and **S3** for data movement, then implemented retries and dead-letter strategies to prevent silent data loss.
- Created batch feature-generation jobs and a metadata layer in **PostgreSQL 12–13**, adding indexes and query refactors when feature lookups became a bottleneck.
- Implemented CI checks for model artifacts and API contracts, catching breaking schema changes before release rather than during late-stage QA.
- Built monitoring dashboards in Prometheus/Grafana, then added alert thresholds tied to model accuracy proxies and input drift indicators.
- Added A/B experimentation support for model variants, allowing safe comparison of new architectures without forcing full cutovers.
- Worked closely with product and data stakeholders to translate vague requirements into measurable acceptance criteria and stable delivery milestones.

Software Developer
October 2015 to May 2018
Datamart

- Built early ML-assisted reporting workflows in **Python 3.5–3.7** and SQL, improving reliability by replacing fragile Excel-based steps with repeatable pipelines.
- Maintained ETL jobs where late-arriving data caused inconsistent aggregates, and fixed it by implementing watermarking, reprocessing windows, and reconciliation reports.
- Introduced **Airflow 1.10** scheduling to replace cron sprawl, adding retries and alerting that reduced "silent failure" incidents across nightly runs.
- Implemented baseline models with **scikit-learn 0.18–0.20** and **TensorFlow 1.x**, documenting feature assumptions so future upgrades didn't break training semantics.
- Migrated legacy stored procedures into a clearer schema design, adding indexes and query tuning after profiling showed repeated full scans under peak report loads.
- Resolved production bugs from timezone and locale edge cases by standardizing UTC storage, validating inputs, and adding regression tests for boundary dates.
- Collaborated with analysts to define data dictionaries and source-of-truth metrics, preventing repeated disputes about KPI definitions across teams.
- Automated packaging and deployment steps with simple CI scripts, reducing manual release errors and making rollbacks predictable.
- Supported incident response for failed jobs by adding structured logs and runbooks, turning tribal knowledge into repeatable operational practice.

EDUCATION

Bachelor's degree in Computer Science
2011 to 2015
University of Amsterdam

- Built strong foundations in software engineering, data structures, and systems design, with coursework that supports designing reliable services and data-driven applications.

PROJECTS

RAG Ops Toolkit

- Built a reusable **RAG** blueprint with ingestion, chunking, embedding refresh, and evaluation gates, designed to drop into new products without rewriting the whole pipeline.
- Added cost/quality telemetry (tokens, latency, recall proxies) and automated canary rollouts so teams can ship LLM features safely while controlling spend.